

Moving from Conda to uv

A simple guide to installing uv, creating Python projects, managing packages, and understanding virtual environments.

1. What is uv?

uv is a fast Python package and environment manager. It can replace many common uses of **Conda + pip + virtualenv** for regular Python projects.

Main idea: each project normally has its own `.venv`, plus a `pyproject.toml` and `uv.lock`.

2. Install uv

Windows

Open **PowerShell** and run:

```
powershell -ExecutionPolicy ByPass  
-c "irm  
https://astral.sh/uv/install.ps1 |  
iex"
```

Then close and reopen PowerShell.

macOS

Open **Terminal** and run:

```
curl -LsSf  
https://astral.sh/uv/install.sh |  
sh
```

Then close and reopen Terminal.

Check that it worked

```
uv --version
```

3. Conda → uv Cheat Sheet

Task	Conda	uv
Create project/environment	<code>conda create -n myproj python=3.12</code>	<code>uv init --python 3.12</code>
Activate environment	<code>conda activate myproj</code>	Usually not needed
Add a package	<code>conda install pandas</code>	<code>uv add pandas</code>
Remove a package	<code>conda remove pandas</code>	<code>uv remove pandas</code>
Run Python	<code>python</code>	<code>uv run python</code>
Run script	<code>python app.py</code>	<code>uv run python app.py</code>
Rebuild environment	<code>conda env create ...</code>	<code>uv sync</code>
Install Python	Managed by Conda	<code>uv python install 3.12</code>
List installed packages	<code>conda list</code>	<code>uv pip list</code>

4. Recommended workflow for a new project

```
mkdir my-project
cd my-project

uv init --python 3.12
uv add numpy pandas scikit-learn matplotlib
uv add --dev pytest ruff ipykernel

uv run python
```

Prefer `uv add package-name` instead of `pip install package-name`. This keeps your project dependencies properly recorded.

5. Important project files

```
my-project/
├──
├── .venv/
├── .python-version
├── pyproject.toml
├── uv.lock
└── main.py
```

- **.venv/** → the virtual environment for this project
- **pyproject.toml** → your main project and dependency file
- **uv.lock** → exact package versions used by the project

Commit to Git:

```
pyproject.toml
uv.lock
.python-version
```

Do not commit:

```
.venv/
```

6. What about 10 projects = 10 virtual environments?

Yes. If you have 10 separate projects, you normally have 10 separate `.venv` folders. This is intentional because each project is isolated.

It is not as wasteful as it looks. `uv` keeps a **global package cache** and can reuse package data efficiently instead of downloading everything from scratch for every project.

Why isolation is good: upgrading NumPy, PyTorch, or pandas for Project B will not accidentally break Project A.

7. Jupyter / VS Code / Cursor

Add Jupyter support:

```
uv add --dev jupyterlab ipykernel
```

Run Jupyter:

```
uv run jupyter lab
```

In VS Code or Cursor, choose the Python interpreter inside:

```
your-project/.venv
```

8. Common data science setup

```
uv add numpy pandas scipy scikit-learn statsmodels matplotlib plotly  
uv add --dev jupyterlab ipykernel pytest ruff
```

9. Removing packages and resetting the environment

Remove a package

```
uv remove pandas
```

This removes the package from your project dependencies and updates the environment.

Delete the entire virtual environment

Windows PowerShell:

```
Remove-Item -Recurse -Force .venv
```

macOS / Linux:

```
rm -rf .venv
```

Deleting `.venv` does **not** delete your project or dependency list. Your dependencies are still recorded in `pyproject.toml` and `uv.lock`.

If your environment becomes broken or confusing, delete `.venv` and run `uv sync` to rebuild it cleanly.

10. When do you use `uv sync`?

`uv sync` makes your local `.venv` match the dependencies defined by the project and lock file.

Use case 1: You cloned a project from GitHub

```
git clone https://github.com/example/my-project.git
cd my-project
uv sync
```

`uv` reads `pyproject.toml` and `uv.lock`, creates the virtual environment, and installs the correct packages.

Use case 2: You deleted `.venv`

```
uv sync
```

`uv` rebuilds the environment from the project files.

Use case 3: Someone else changed dependencies

```
git pull
uv sync
```

This updates your local environment to match the latest project definition.

Think of `uv sync` as: **"Make my computer match this project's dependency recipe."**

11. The 8 commands to remember

```
uv init
uv add pandas
uv remove pandas
uv sync
uv run python
uv run python app.py
uv python install 3.12
uv python list
```

12. Simple rules

1. One real project → one `.venv`.
2. Use `uv add` instead of `pip install`.
3. Use `uv run` instead of relying on environment activation.
4. Commit `uv.lock`.
5. Do not commit `.venv`.
6. Use `--dev` for Jupyter, testing, and development tools.
7. If the environment gets messy, delete `.venv` and run `uv sync`.
8. Keep Conda only when you truly need unusual non-Python/system packages.

Bonus: instruction for AI coding assistants

You can place this in `CLAUDE.md` or `AGENTS.md`:

```
This project uses uv for Python dependency and environment management.
Use `uv add` / `uv remove` for dependencies and `uv run` for commands.
Do not use pip or Conda unless explicitly requested.
```